01 数据结构基础

- 数据结构概念
- 抽象数据类型
- 算法性能分析

01 数据结构基础

- 数据结构概念
- 抽象数据类型
- 算法性能分析

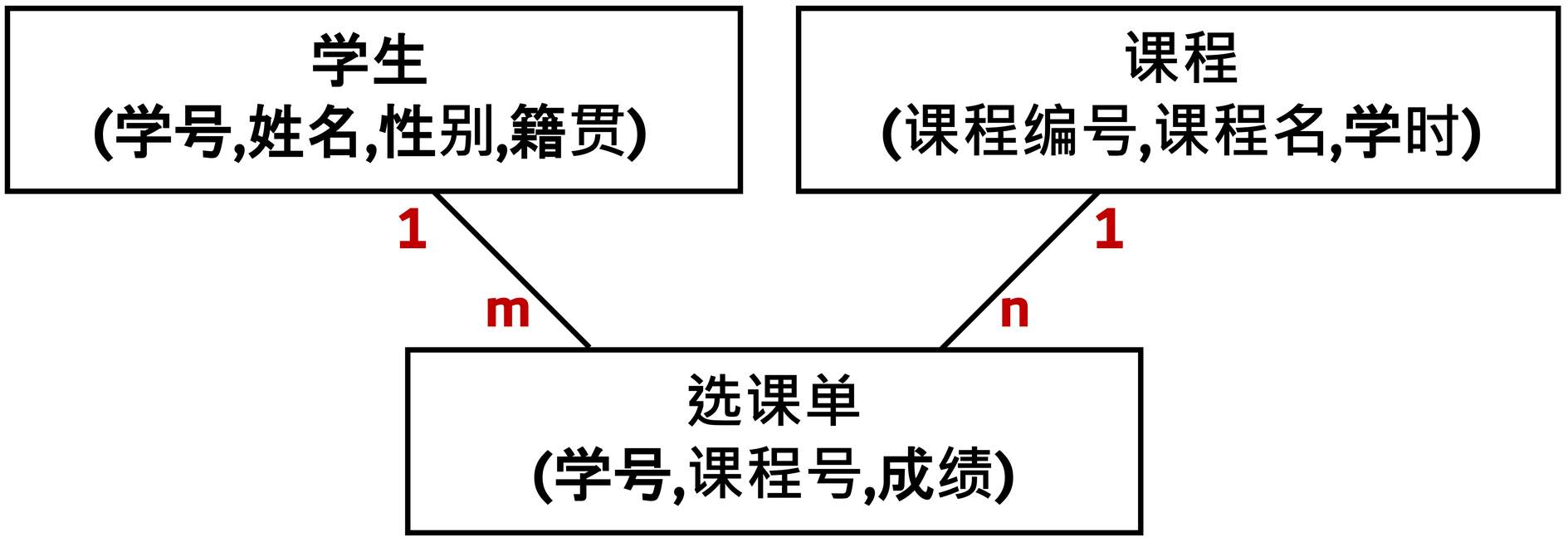
“学生”表格

学 号	姓　名	性别	籍　贯	出生年月
98131	刘激扬	男	北　京	1979.12
98164	衣春生	男	青　岛	1979.07
98165	卢声凯	男	天　津	1981.02
98182	袁秋慧	女	广　州	1980.10
98224	洪　伟	男	太　原	1981.01
98236	熊南燕	女	苏　州	1980.03
98297	宫　力	男	北　京	1981.01
98310	蔡晓莉	女	昆　明	1981.02
98318	陈　健	男	杭　州	1979.12

“课程”表格

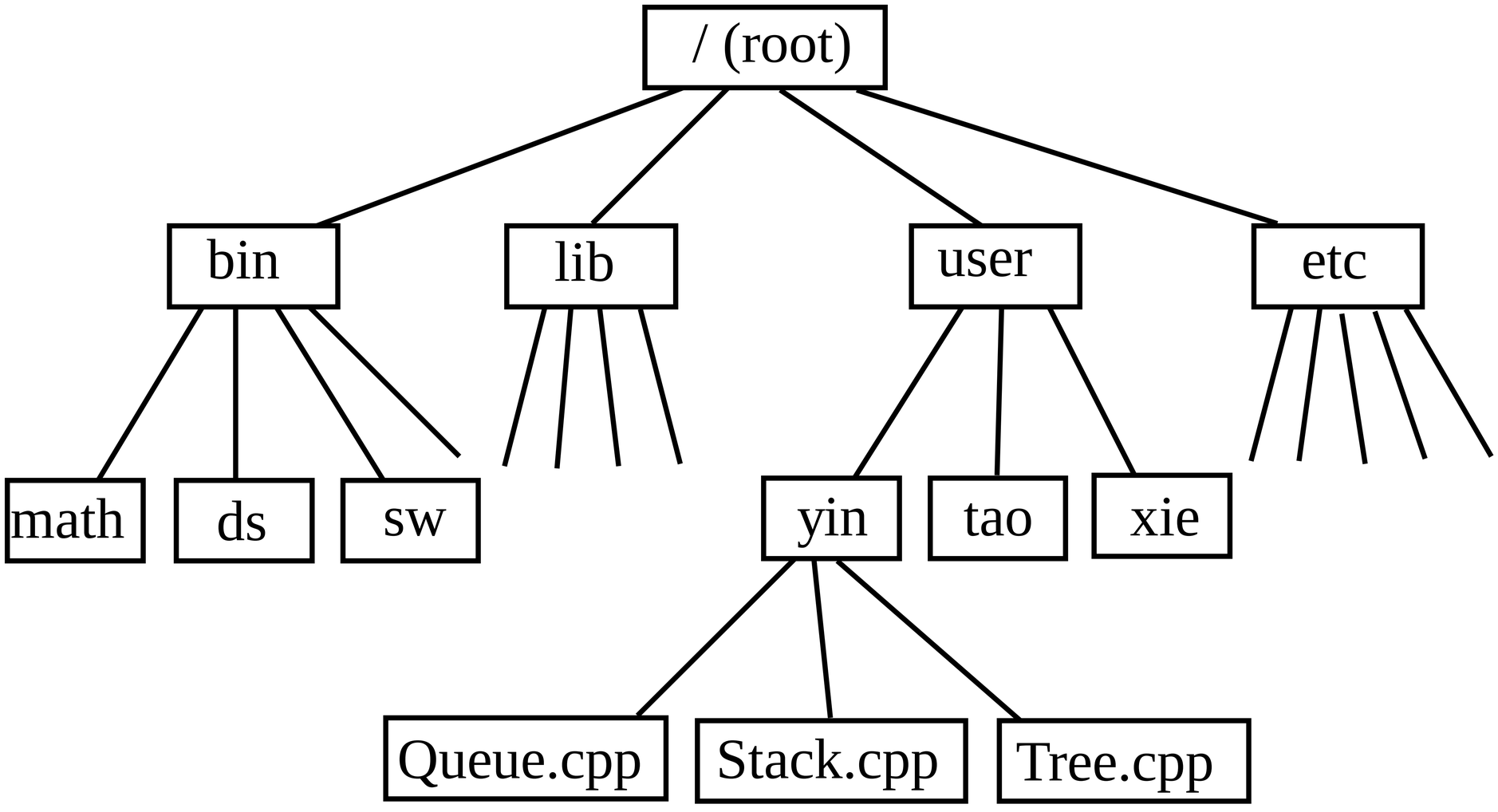
课程编号	课 程 名	学时
024002	程序设计基础	64
024010	汇编语言	48
024016	计算机原理	64
024020	数据结构	64
024021	微机技术	64
024024	操作系统	48
024026	数据库原理	48

选课单



学生选课系统中实体间构成的网状关系

UNIX文件系统的系统结构图



生活中到处都是数据结构

- 吃饭排队是数据结构
- 出行线路是数据结构
- 打扑克牌是数据结构
- …

数据(Data)

- 数据是信息的载体
  - 数据是描述客观事物的数、字符、以及所有能输入到计算机中并被计算机程序识别和处理的符号的集合

- 数值性数据
  - 例如，整数、浮点数等

- 非数值性数据
  - 例如，字符、字符串、文字、图形、图像、语音、代码等

数据元素(Data Element)

- 数据元素是**数据的基本单位**，在计算机程序中常作为一个整体进行访问和处理

- 一个数据元素可以由若干个**数据项**(Data Item)组成，数据项是具有独立含义的最小标识单位

- 数据元素又称为元素、结点、记录

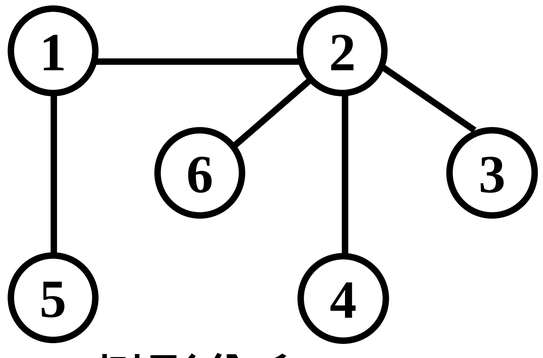
数据对象(Data Object)

- 数据的子集，即具有相同性质的数据元素的集合
  - 例如，学生数据对象

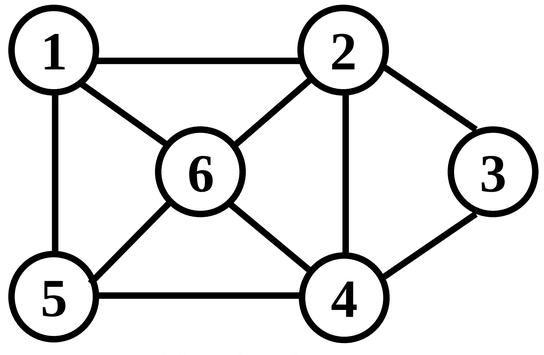
- 数据对象中所有数据元素之间存在**某种关系**
  - 例如，学生按学号排序

数据结构(Data Structure)

- 数据结构由某一**数据对象**(记为D)及该对象中**数据元素之间的关系**(记为R)组成，记为DS={D, R}
  - 例如，n个网站之间的连通关系



树形关系

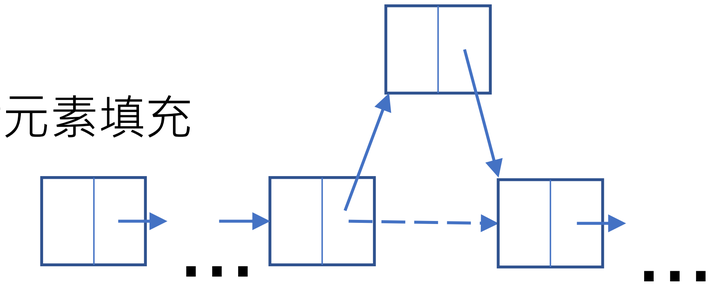


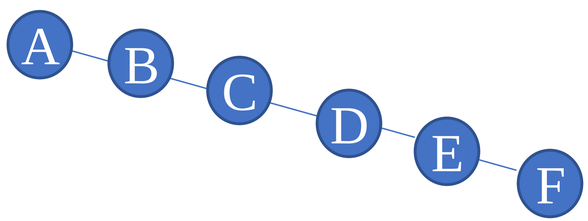
网状关系

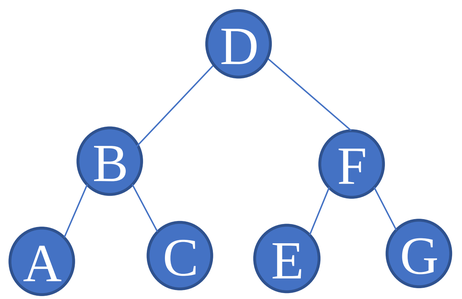
- 数据结构(技术)就是根据各种不同的数据对象和数据元素之间的关系，研究如何**表示**、**存储**、以及**操作**(查找、插入、删除、修改、排序)这些数据元素的技术

数据结构在程序设计中的作用

- **程序 = 数据结构 + 算法**
- **以编制一个通讯录应用程序为例**
  - **解决方案1：顺序表**
    - **算法**：顺序查找，末尾增加，删除元素用最后元素填充
    - **缺点**：联系人多了以后，查找效率低
  - **解决方案2：有序顺序表**
    - **算法**：用折半搜索提高查询效率
    - **缺点**：查询效率高了，但插入效率又降低了
  - **解决方案3：线性链表**
    - **插入效率高了，但查询效率又下降了**
  - **解决方案4：二叉搜索树**
    - 查询效率可望达到log(n)，**插入**删除也快
    - **缺点：如果生成的是**单枝树，就是线性链表
  - **解决方案5：高度平衡的二叉搜索树**
  - **……**







数据结构的三个方面

- 数据的逻辑结构，即数据元素间的逻辑关系

- 数据的存储结构，即数据元素及其关系在计算机存储内的表示

- 数据的运算，即对数据元素施加的操作

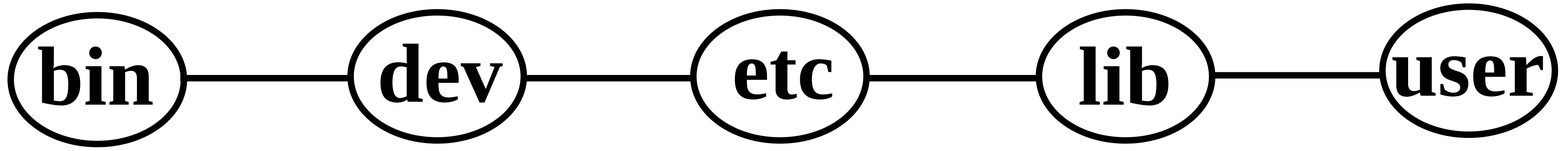
数据的逻辑结构

- 数据的逻辑结构从逻辑关系上描述数据，**与数据元素的存储、存储位置无关**

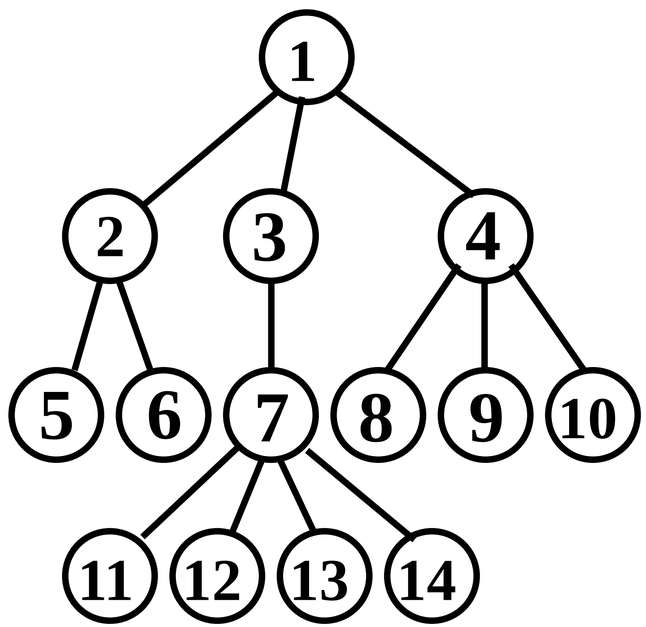
- 数据的逻辑结构可以看作是**从具体问题抽象出来的数据模型**

- 数据的逻辑结构分类
  - 线性结构，例如，线性表
  - 非线性结构，例如，树、图

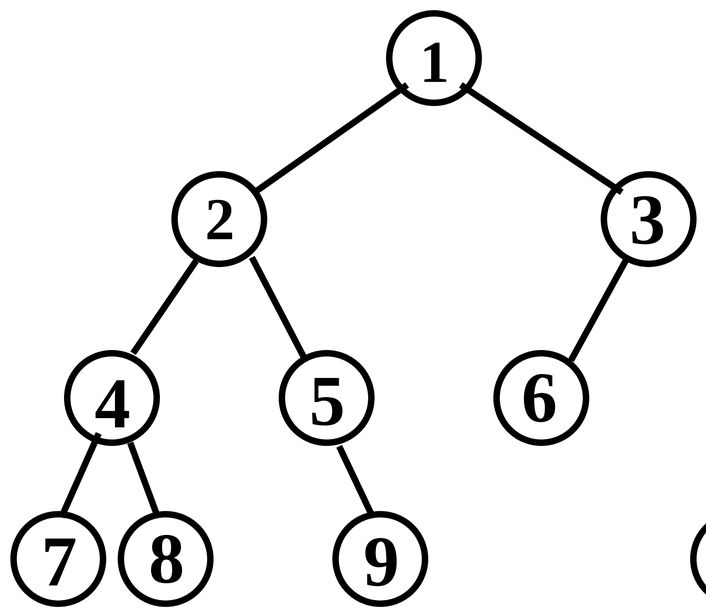
线性结构



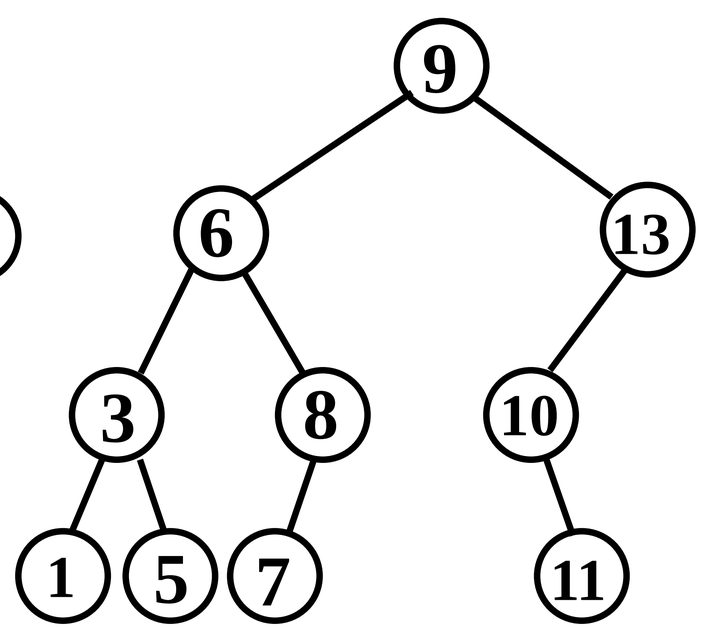
树形结构



树

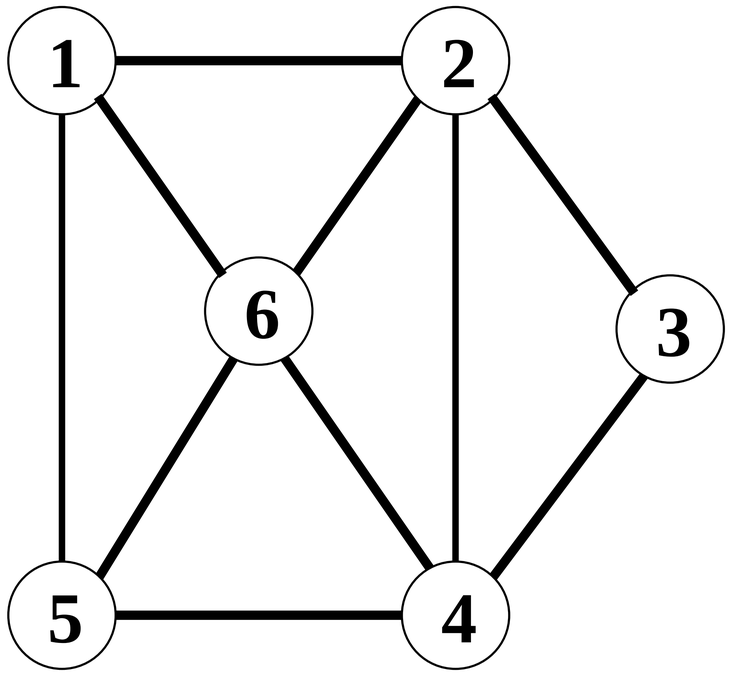


二叉树

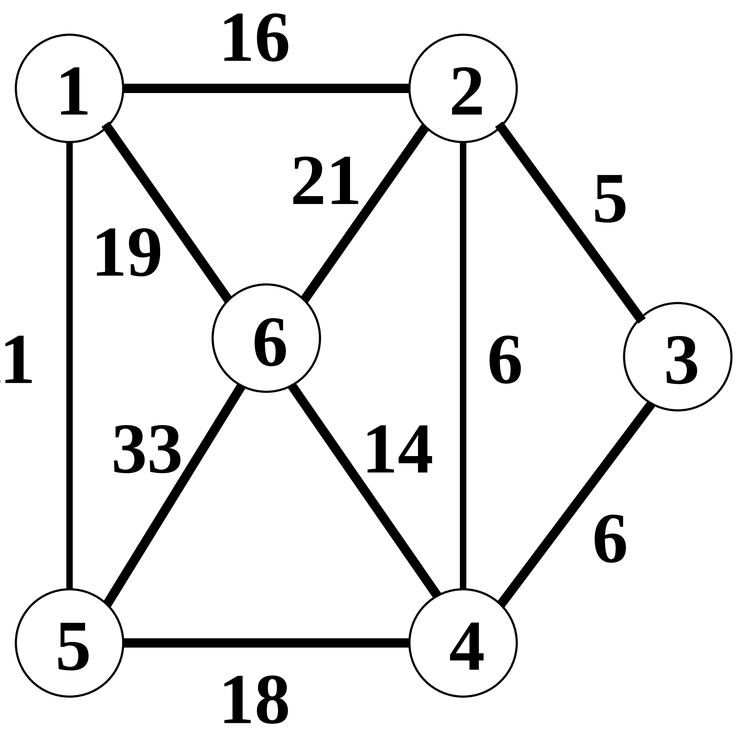


二叉搜索树

图结构



网络结构



数据的存储结构

- 数据的存储结构是逻辑结构用计算机语言的实现

- 数据的存储结构分类
  - 顺序存储表示(Sequential Storage)
  - 链接存储表示(Linked Storage)

  } 主要用于内存的存储表示

  - 索引存储表示(Indexed Storage)
  - 散列存储表示(Hashing Storage)

  } 主要用于外存的存储表示

数据的运算

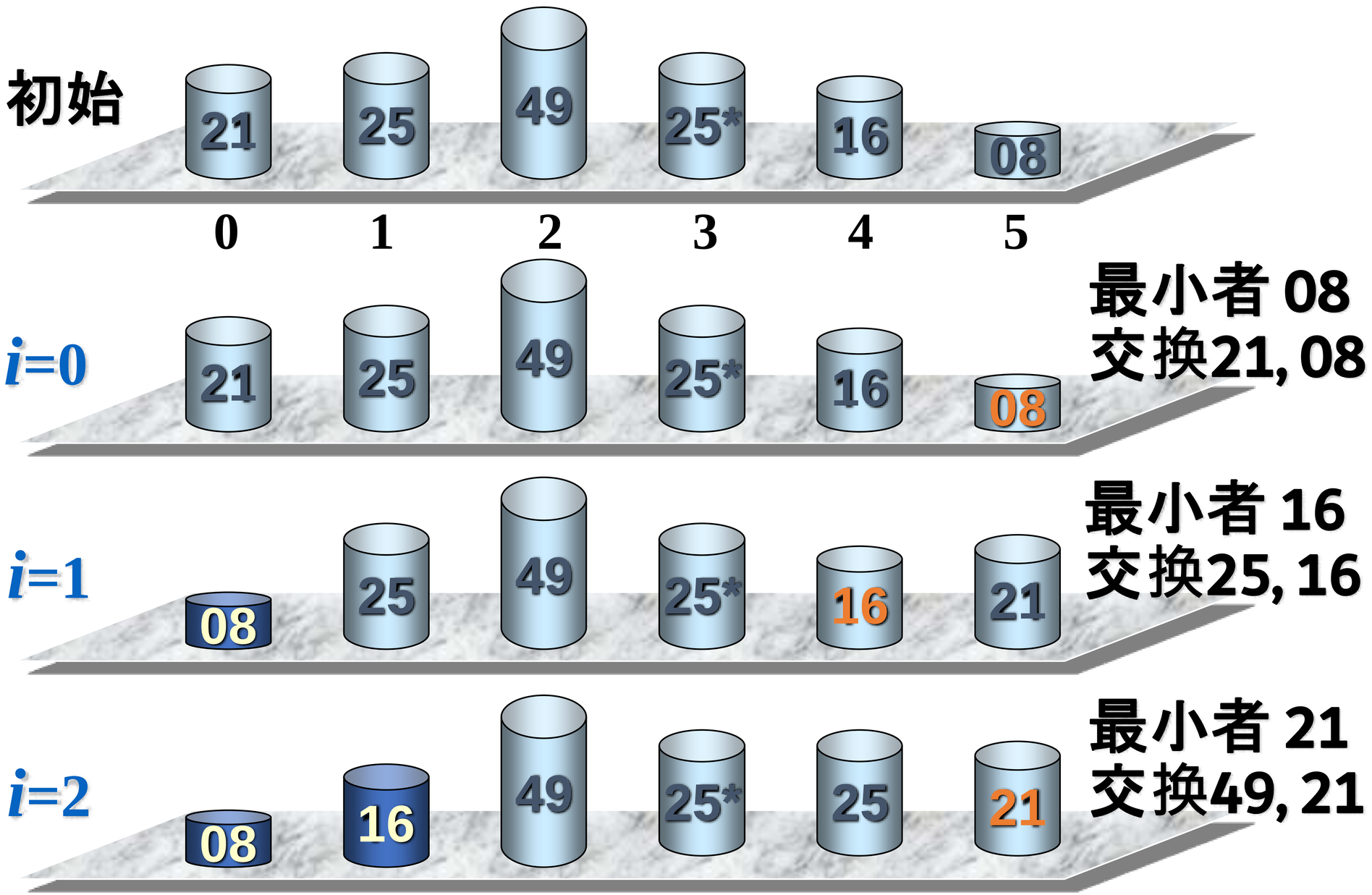
- **算法**：一个有穷的指令集，这些指令为解决某一特定任务规定了一个运算序列
  - 有输入：有0个或多个输入
  - 有输出：有一个或多个输出(计算结果)
  - 确定性：每步定义都是确切无歧义的
  - 有穷性：应该在执行有穷步后结束
  - 能行性：每一条运算都必须是足够基本的，能通过计算机指令精确地执行

算法设计：自顶向下、逐步求精

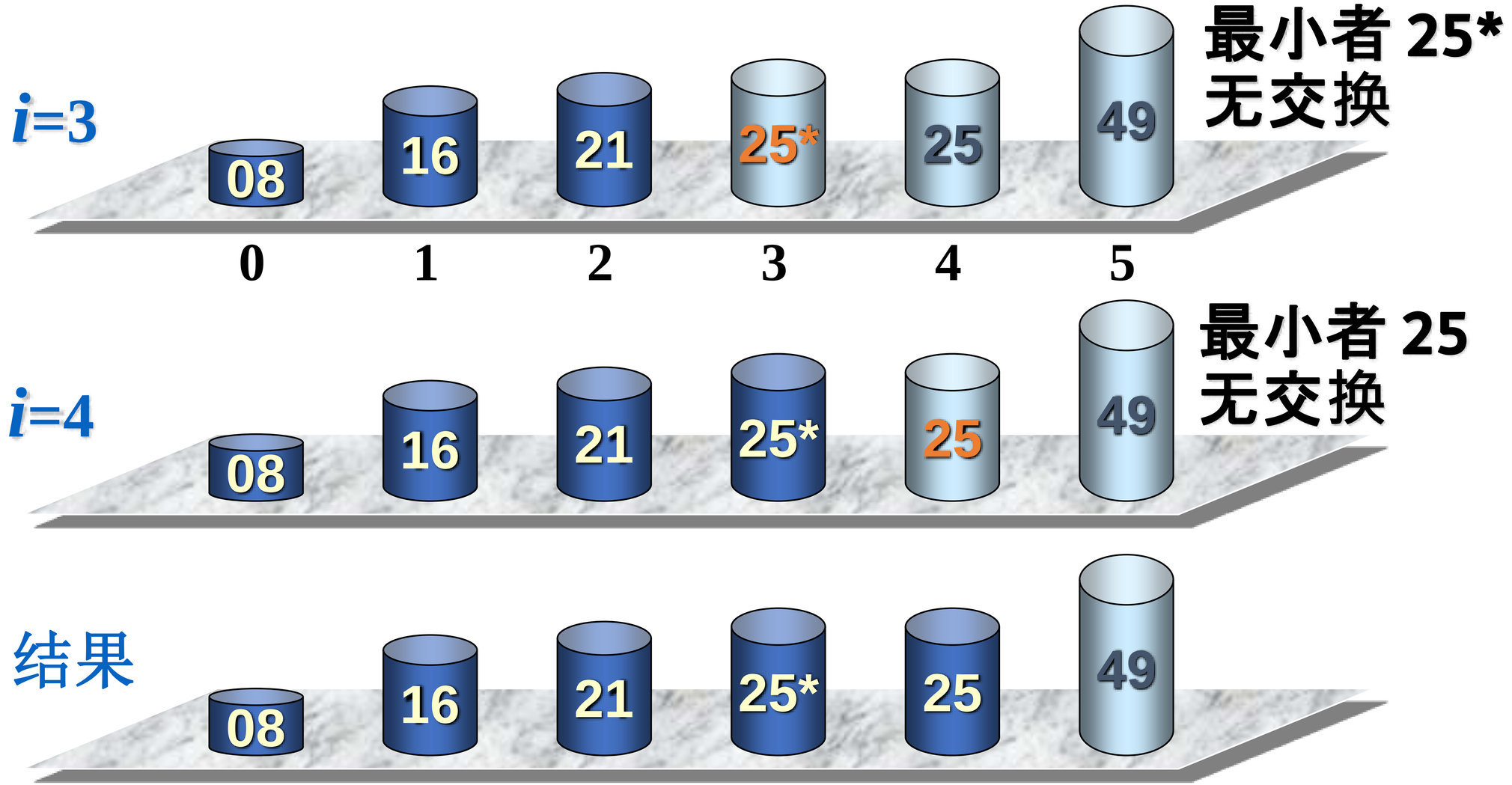
- 事例学习：选择排序问题
- 明确问题：递增排序
- 解决方案：逐个选择最小数据
- 算法框架：

```
for (int i = 0; i < n-1; i++ ) {    // n-1趟
        从a[i]检查到a[n-1];
        若最小整数在a[k], 交换a[i]与a[k];
}
```

各趟排序后的结果


初始
21
25
49
25*
16
08
0
1
2
3
4
5
i=0
21
25
49
25*
16
08
最小者 08
交换21, 08
i=1
08
25
49
25*
16
21
最小者 16
交换25, 16
i=2
08
16
49
25*
25
21
最小者 21
交换49, 21

各趟排序后的结果


i=3
08
16
21
25*
25
49
最小者 25*
无交换
0
1
2
3
4
5
i=4
08
16
21
25*
25
49
最小者 25
无交换
结果
08
16
21
25*
25
49

算法设计：自顶向下、逐步求精

- 细化程序：程序**selectSort**

```
void selectSort(int a[ ], const int n) {
    // 对n个整数a[0]到a[n-1]按递增顺序排序
    for (int i = 0; i < n - 1; i++) {
        int k = i;
        // 从a[i]查到a[n-1]，找最小整数，在a[k]
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[k])  k = j;
        int temp = a[i];  a[i] = a[k];  a[k] = temp;
    }
}
```

数据结构的课程内容体系

层次	数据表示	数据处理
抽象	逻辑结构	运算
实现	存储结构	算法
评价	不同数据结构的比较与算法分析	

01 数据结构基础

- 数据结构概念
- **抽象数据**类型
- 算法性能分析

数据类型

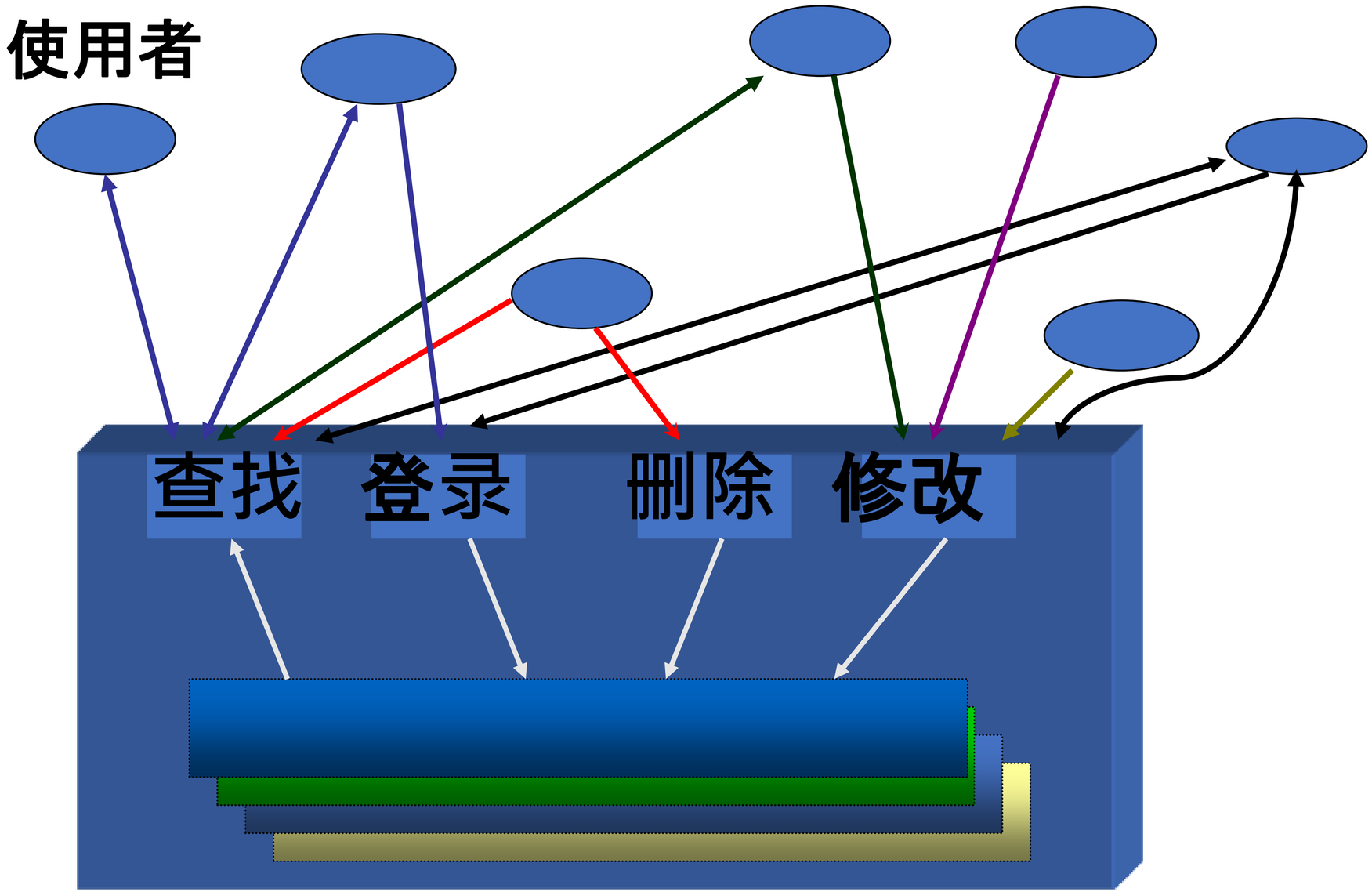
- **数据**类型：一组性质相同的值的集合、以及定义于这个值集合上的一组操作的总称
  - **基本数据**类型：例如，字符型、整型、浮点型等
  - **构造数据**类型：由基本数据类型或构造数据类型组成

- **数据**类型就是数据结构，是从编程者的角度来使用的

抽象数据类型(Abstract Data Type)

- 抽象数据类型是数据逻辑特性的表示，与在计算机内的表示和实现细节无关，是数据结构的抽象形式
  - 由用户定义，用以表示应用问题的数据模型
  - 由基本的数据类型组成, 并包括一组相关的服务(或称操作)
  - 信息隐蔽、数据封装、使用与实现相分离
- 抽象数据类型概念进一步发展：面向对象概念

抽象数据类型(Abstract Data Type)



自然数的抽象数据类型定义

ADT NaturalNumber is

objects: 整数的有序子集合，开始于0，结束于机器能表示的最大整数MaxInt

functions: 对于x, y ∈ NaturalNumber，+、-、<、==、=等都是可用的服务

Zero(): NaturalNumber	**返回自然数0**
IsZero(x): Boolean	**if (x == 0) 返回True else 返回False**
Add(x, y): NaturalNumber	**if (x+y <= MaxInt) 返回x+y else 返回MaxInt**
Sub(x, y): NaturalNumber	**if (x < y) 返回0 else 返回x - y**
Equal(x, y): Boolean	**if (x==y) 返回True else 返回False**
Successor(x): NaturalNumber	**if (x==MaxInt) 返回x else 返回x + 1**

后继

01 数据结构基础

- 数据结构概念
- 抽象数据类型
- 算法性能分析

算法性能分析

- 算法的性能标准
- 算法的后期测试
- 算法的事前估计
- 算法的渐进分析

算法的性能标准

- 正确性
- 可使用性(用户友好性)
  - 良好的界面、完备的用户文档
- 可读性
  - 理解、测试、修改算法的基础
  - 代码风格(例如，命名和注释)
- 效率
  - 计算资源(例如，时间和空间)的消耗
- 健壮性(容错性)
- …

算法的后期测试

- 在算法中的某些部位插装时间函数**time()**来测量算法完成某一功能所花费的时间
  - 测量顺序搜索(Sequential Search)的执行时间

```
int seqSearch (int a[ ], const int n, const int x) {
    int i = 0;
    while (i < n && a[i] != x) i++;
    if (i==n) return -1;
    return i;
}
```

```
double start, stop;
time(&start);
int k = seqSearch(a, n, x);
time(&stop);
double runTime = stop - start;
cout<<n<<“ ”<<runTime<<endl;
```

- time()是毫秒级，对少于毫秒的数量级，计时不准，可把被测部分放在循环中多次运行，除以运行次数得到每次运行的平均时间

算法的后期测试的缺点

- 算法的运行时间依赖于具体环境配置
  - 在速度不同的计算机上，算法执行速度相差非常大
  - 可用存储空间不够，算法需要的运行时间很多
  - 不同的编译器编译出的目标代码不一样长，所需执行时间不同

- 算法的后期测试主要用于评估算法的正确性和可用性，并不能判断算法的效率优劣
  - 通过比较算法的复杂性来评价
  - 算法复杂性与具体运行环境配置无关

算法的事前估计

- 用来确定问题规模n与算法实现时需要的时间开销T(n)、以及空间开销S(n)的关系
  - 时间复杂度T(n)
  - 空间复杂度S(n)

- 对算法进行分析时，时空复杂性是最重要的基本功，是最理想的算法效率评价标准

空间复杂度度量

- 存储空间固定部分(静态空间)
  - 存放程序指令代码的空间，常数、简单变量、定长成分（如数组元素、结构成分、对象的数据成员等）变量所占空间
- 存储空间可变部分(动态空间)
  - 与实例特性有关的变量所占空间、递归工作栈所用空间、以及通过new和delete命令动态使用的空间

时间复杂度度量

- 编译时间：与编译程序有关，与实例性质无关

- 运行时间：可以用**程序步**度量
  - 语法或语义上有意义的指令序列，执行时间与实例特性无关
  - 例如，**声明语句**的程序步数为**0**，**表达式**的程序步数为**1**

程序步确定方法

- **方法1：插入计数全局变量count**
- **方法2：建表，列出各语句的程序步**

例1：以迭代方式求累加和

```
float sum (float a[ ], int n) {
    float s = 0.0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```

在求累加和程序中加入count语句

```
float sum (float a[ ], int n) {
    float s = 0.0;
    count++;  // 针对赋值语句
    for (int i = 0; i < n; i++) {
        count+=2;  // 针对for语句
        s += a[i];
        count++;  // 针对赋值语句
    }
    count+=2;  // 针对for的最后一次迭代
    count++;  // 针对return语句
    return s;
}
```

执行结束得到程序步数：**count=3n+4**

注意

- **一个语句本身的程序步数，可能不等于该语句一次执行所具有的程序步数**

- **例如：赋值语句x = sum(R, n)**
  - **本身的程序步数为1**
  - **一次执行对函数sum(R, n)的调用需要的程序步数为3n+4**
  - **一次执行该赋值语句的程序步数为1+3n+4 = 3n+5**

为求累加和程序建立程序步计算表格

程序语句	一次执行所需程序步数	执行频度	执行步数
float s = 0.0;	1	1	1
for (int i = 0; i < n; i++)	2	n+1	2n+2
s += a[i];	1	n	n
return s;	1	1	1
	总程序步数		**3n+4**

例2：以递归方式求累加和

```
float rsum (float a[ ], int n) {
    if (n <= 0)
        return 0;
    else
        return rsum(a, n-1) + a[n-1];
}
```

在递归求累加和程序中加入count语句

```
float rsum(float a[ ], int n) {
    count++;  // 针对if语句
    if (n <= 0) {
        count++;  // 针对return语句
        return 0;
    } else {
        count += 2;  // 针对return语句
        return rsum(a, n-1) + a[n-1];
    }
}
```

①表达式
②return

在递归求累加和程序中加入count语句

- 设count初始值为0，$T_{rsum}(n)$是程序执行后的count值

$n = 0, T_{rsum}(0) = 2$

$n > 0, T_{rsum}(n) = 3 + T_{rsum}(n-1)$

$$
\begin{aligned}
T_{rsum}(n) &= 3 + T_{rsum}(n-1) \\
&= 3 + 3 + T_{rsum}(n-2) = 3*2 + T_{rsum}(n-2) \\
&= 3 + 3 + 3 + T_{rsum}(n-3) = 3*3 + T_{rsum}(n-3) \\
&= \cdots\cdots \\
&= 3n + T_{rsum}(0) \\
&= 3n + 2
\end{aligned}
$$

为递归求累加和程序建立程序步计算表格

程序语句	一次执行所需执行步数	执行频度		执行步数	
		n=0	n>0	n=0	n>0
if (n <= 0)	1	1	1	1	1
return 0;	1	1	0	1	0
else return rsum(a, n-1) + a[n-1];	$2 + T_{rsum}(n-1)$	0	1	0	$2 + T_{rsum}(n-1)$
	总程序步数			2	$3 + T_{rsum}(n-1)$

程序步

- **sum比rsum慢吗？**

- **程序步不是一个准确的概念，而是一个对每条语句运行时间的抽象概念**

- **再作一次抽象，从由多种因素构成的时间复杂性中抽取出主要因素，将常数抽象为1**
  - **有利于抓住主要矛盾，简化复杂性分析**

算法的渐进分析

- 若n是问题规模，则时间复杂度T(n)是问题规模n的函数

- $T_{p1}(n)=c_1n^2+c_2n$和$T_{p2}(n)=c_3n$，哪个快（$c_1, c_2, c_3 > 0$）？
  - $T_{p1}(n)$是n^2数量级，而$T_{p2}(n)$是n数量级
  - 存在n_0，使得$T_{p1}(n) > T_{p2}(n)$对所有的$n>n_0$成立

- 当n趋于无穷大时，把时间复杂度的**数量级**称为算法的渐进时间复杂度
  - 大O表示法：表示算法时间复杂度的上限
  - Ω表示法：表示算法时间复杂度的下限
  - Θ表示法：表示算法时间复杂度的上下限相等

大O表示法

- 若存在正整数$c > 0$和$n_0 > 0$，使得$T(n) \leq c*f(n)$对所有的$n \geq n_0$成立，则称该算法的渐进时间复杂度在$O(f(n))$中，记为$T(n) = O(f(n))$

- $T(n) = 3n + 2$
  - 当$n \geq 2$时，$3n + 2 \leq 3n + n = 4n$，所以$T(n) = O(n)$
- $T(n) = 10n^2 + 4n + 2$
  - 当$n \geq 2$时，$10n^2 + 4n + 2 \leq 10n^2 + 4n + n = 10n^2 + 5n$
  - 当$n \geq 5$时，$10n^2 + 5n \leq 10n^2 + n^2 = 11n^2$，所以$T(n) = O(n^2)$
- $T(n) = 9$
  - 当$n_0 = 1$，$c = 9$时，$T(n) = O(1)$

大O表示法的加法规则

- 若$T_1(m) = O(f(m))$，$T_2(n) = O(g(n))$
  则$T_1(m) + T_2(n) = O(max(f(m), g(n)))$

- $c < log_2n < n < nlog_2n < n^2 < n^3 < 2^n < 3^n < n!$
  - 取c、log^2n、n、$nlog^2n$，时间效率比较高
  - 取n^2、n^3，时间效率差强人意
  - 取2^n、3^n、n!，当n稍微大一点，算法的时间代价变为很大，以至于不能计算

- $T_1(n) = O(n)$，$T_2(n) = O(n^2)$，则$T_1(n) + T_2(n) = O(n^2)$

大O表示法的乘法规则

- 若$T_1(m) = O(f(m))$，$T_2(n) = O(g(n))$
  则$T_1(m) * T_2(n) = O(f(m) * g(n))$

- $T_1(n) = O(n)$，$T_2(n) = O(n^2)$，则$T_1(n) * T_2(n) = O(n^3)$

大O表示法的简化规则

- 若$T(n) = a_m n^m + \ldots + a_1 n + a_0$，$a_m > 0$
  则$T(n) = O(n^m)$

- $T(n) = 2n^{10} + 10n^9$，则$T(n) = O(n^{10})$

求两个n阶方阵的乘积C = A x B

```
void MatrixMultiply (int A[n][n], int B[n][n], int C[n][n]) {
    for (int i=0; i<n; i++)                                  … 2(n+1)
        for (int j=0; j<n; j++) {                            … 2n(n+1)
            C[i][j]=0;                                       … n^2
            for (int k=0; k<n; k++)                          … 2n^2(n+1)
                C[i][j]=C[i][j]+A[i][k]*B[k][j];  … n^3
        }
}
```

每次2步 n+1次
每次2步 n(n+1)次
每次1步 n^2次
每次2步 n^2(n+1)次
每次1步 n^3次

- 算法中所有语句的频度之和是矩阵阶数n的函数
  - $T(n) = 3n^3+5n^2+4n+2 = O(n^3)$
  - 启发？

大O表示法的使用

- 需要考虑关键操作的程序步数
- 关键操作大多在循环和递归中
- 可直接考虑关键操作的执行频度，导出其与问题规模n的函数关系f(n)

大O表示法的使用

- 单个循环
  - 循环内的简单语句即为关键操作，该程序段的渐进时间复杂度应是此关键操作的执行频度的大O表示
- 几个并列的循环
  - 分析每个循环的渐进时间复杂度，然后利用大O表示法的加法规则来计算渐进时间复杂度
- 多层的嵌套循环
  - 关键操作应该在最内层循环中，先自外向内层层分析每层循环的时间渐进复杂度，然后利用大O表示法的乘法规则来计算渐进时间复杂度

例子：变量计数

```
x=0;  y=0;
```

$T_1(n)=O(1)$

```
for (int k=0; k<n; k++)
    x++;
```

$T_2(n)=O(n)$

```
for (int i=0; i<n; i++)
    for (int j=0; j<n; j++)
        y++;
```

$T_3(n)=O(n^2)$

$$T(n) = T_1(n) + T_2(n) + T_3(n) = O(\max(1, n, n^2)) = O(n^2)$$

例子：两个并列循环

```
void exam(float x[ ][ ], int m, int n) {
    float sum[ ];
    for (int i=0; i<m; i++) {  // x中各行
        sum[i]=0.0;  // 数据累加
        for (int j=0; j<n; j++)
            sum[i]+=x[i][j];
    }
    for (i=0; i<m; i++)  // 打印各行数据和
        cout<<i<<“: ”<<sum[i]<<endl;
}
```

O(max(m*n, m)) = O(m*n)

例子：两个嵌套循环

- **冒泡排序**
  - 设待排序对象序列中的对象个数为**n**
  - **作n-1趟，i=1, 2, …, n-1**
  - **在第i趟中从后向前，j=n-1, n-2, …, i，顺次两两比较，如果发生逆序，则互相交换**

```
BubbleSort( ) {
    int i=1;
    while (i<ArraySize) {
        BubbleExchange(i);
        i++;
    } // 共n-1趟比较
}
```

```
BubbleExchange(int i) {
    // 共n-i次比较
    for (int j=ArraySize-1; j>=i; j--)
        if (Element[j-1]>Element[j]) {
            Swap(j-1, j);
        }
}
```

- **O(f(n)*g(n)) = O(n*n) = O(n^2)**

（ $\because \sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$ ）

1+2+…+(n-1)

Ω和Θ表示法

- 若存在正整数$c > 0$和$n_0 > 0$，使得$T(n) \geq c*f(n)$对所有的$n \geq n_0$成立，则称该算法的渐进时间复杂度在$\Omega(f(n))$中，记为$T(n) = \Omega(f(n))$

- 若$T(n) = O(f(n))$且$T(n) = \Omega(f(n))$，则$T(n) = \Theta(f(n))$

最坏、最好和平均情况

```
int seqSearch (int a[ ], const int n, const int x) {
    int i = 0;
    while (i < n && a[i] != x) i++;
    if (i==n) return -1;
    return i;
}
```

- 最好情况：O(1)
- 最坏情况：O(n)
- 平均情况：O(n)

期望 $\frac{1}{n}(1+2+\cdots+n)$

$= \frac{n+1}{2}$

渐进的空间复杂度

- **当问题规模n充分大时，需要的存储空间体积如何变化**
  - 为解决问题所需要的辅助存储空间，例如，交换数据时所需要的临时存储空间
  - **不包括程序指令、常数、指针等所需要的存储空间(这些是固定的)，也不是输入数据所占用的存储空间**

- **大O表示法**
  - **设S(n)是算法渐进空间复杂度，可以表示为n的某个函数f(n)的数量级：S(n)=O(f(n))**

两种代价计算方法的比较

- 事前估计——复杂性分析
  - 更全面地分析程序的执行代价，例如从最坏的情况对程序的代价进行估计，如果知道数据的分布情况还可以对程序执行的平均代价进行估计
  - 一般这种方法难以获得程序的具体执行时间
- 后期测试
  - 可获取程序每次执行所需的时间和空间，这种方法获得的结果是针对某特定的数据和情况下获得
  - 要获得程序的整体执行效率，需要经过多次反复的测试，并精心设计测试数据
- 在实际应用中，往往是通过将两者结合的方式对系统的性能进行分析

练习

- 分析程序段“i=1; while(i<=n) i=i*2;”的时间复杂度

- 循环次数k满足$2^k=n$，$k=\log_2 n$，因此$T(n) = O(\log_2 n)$

练习

- **分析如下递归函数的时间复杂度**

```
Fact(int n) {
    if (n<=1) return 1;
    else return n*Fact(n-1);
}
```

- **T(n) = D, n ≤ 1**
- **T(n) = C + T(n-1), n > 1**

- **T(n) = 2*C + T(n-2) = (n-1)*C + T(1) = (n-1)*C + D = O(n)**

练习

- **分析如下程序段的时间复杂度**

```
for (k=1; k<=n; k*=2) // 执行log2n次
    for (j=1; j<=k; j++) // 执行k次
        sum++;
```

- $T(n) = \sum_{i=0}^{log_2 n} 2^i = 2^{\log_2 n + 1} - 1 = 2n - 1 = O(n)$

$$= \frac{1 - 2^{\log_2 n + 1}}{1 - 2}$$

本章小结

- 数据结构概念
- 抽象数据类型
- 算法性能分析